# Reviewer Pack — Minimal Order of Abelian Varieties over Function Fields vs Q…

Entry 4a39ac5d9adf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:12:52 UTC

## Minimal Order of Abelian Varieties over Function Fields vs Quantum Query Complexity for Bell's Theorem

**Entry ID**: `4a39ac5d9adf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 18:12:52 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Abelian Varieties) **Field B** (complexity object): Quantum Computing (Bell's Theorem Query Complexity)

**Statement**:

> ['For every Abelian variety A over a function field K with genus g, the minimal order of an element in its multiplicative group G(K) that satisfies a Bell inequality with k parties is at least αg^2 for some constant α.', 'The quantum query complexity Q_BELL_k(A) for a Bell inequality with k parties involving Abelian variety A is upper bounded by O(αg^2).']

**Rationale (proposer's reasoning)**:

> ['Abelian varieties have been used in the study of number theory and arithmetic geometry, but their application to quantum information theory is novel. The conjecture leverages the algebraic structure of Abelian varieties to predict properties of Bell inequalities, which are central to understanding quantum correlations.']

**Taxonomy category**: `AbelianVarietiesToQuantumComplexity` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `8d98846fe8002346`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if for a given genus g, at least 80% of randomly generated Abelian varieties A have an element in G(K) with order ≥ αg^2 and quantum query complexity Q_BELL_k(A) ≤ O(αg^2), where α is a constant. The conjecture is falsified if any seed generates an Abelian variety A with an element in G(K) having order < αg^2 or Q_BELL_k(A) > O(αg^2).

## 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `Minimal order Abelian varieties function fields AND Bell's Theorem quantum query complexity` - `Abelian varieties genus g multiplicative group G(K) Bell inequality parties` - `Quantum computing Bell's Theorem query complexity upper bound O(alpha g^2) algebraic geometry`

**Top relevant hits considered**: - [http://arxiv.org/abs/1501.05640v1] Bell's theorem tells us NOT what quantum mechanics IS, but what quantum mechanics IS NOT - [http://arxiv.org/abs/1912.05653v4] Minimal abelian varieties of algebras, I - [http://arxiv.org/abs/1502.03923v1] Bringing Bell's theorem back to the domain of Particle Physics & Cosmology - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode–g-mode tidal instability with GW170817 - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/1411.441 Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2504.12989v3] Query Complexity of Classical and Quantum Channel Discrimination - [http://arxiv.org/abs/1109.4165v2] Quantum Query Complexity of Subgraph Containment with Constant-sized Certificates

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_function_field(g):
        # Simplified function field generation for demonstration purposes
        return [random.randint(0, 1) for _ in range(2**g)]

    def find_minimal_order(A, k):
        # Placeholder for finding minimal order of an element satisfying a Bell inequality
        return random.randint(1, len(A))

    def quantum_query_complexity(g):
        # Placeholder for quantum query complexity calculation
        return g * g
```

```
    n = 30
    instances_tested = 0
    total_order = 0
    total_query_complexity = 0

    for _ in range(n):
        g = random.randint(1, 4)
        A = generate_function_field(g)
        k = random.randint(2, 5)

        order = find_minimal_order(A, k)
        query_complexity = quantum_query_complexity(g)

        total_order += order
        total_query_complexity += query_complexity
        instances_tested += 1

    mean_order = total_order / instances_tested
    mean_query_complexity = total_query_complexity / instances_tested

    conjecture_holds = mean_order >= 2 * g**2 and mean_query_complexity <= 2 * g**2
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "order_and_query_complexity",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_order = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
stances_tested': 30, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 4.233333333333333, 'instances_
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 3.7333333333333334, 'instances
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 5.6, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 3.5, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 3.6, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 4.4, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 4.233333333333333, 'instances_
TRIAL: {'metric_name': 'order_and_query_complexity', 'metric_value': 3.8, 'instances_tested': 30, 'c

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f41b26d.py", line 81, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8f41b26d.py", line 81, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 18395 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 6359 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 4785 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 6186 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 29295 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8092 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8370 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7530 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 12387 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 101399 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/4a39ac5d9adf.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4a39ac5d9adf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/4a39ac5d9adf.tar.gz` (if generated)